LAB MANUAL

Application Observability with Prometheus, Grafana and ELK

B.Tech Computer Science & Engineering

Monitoring application performance and health + centralized log collection and analysis

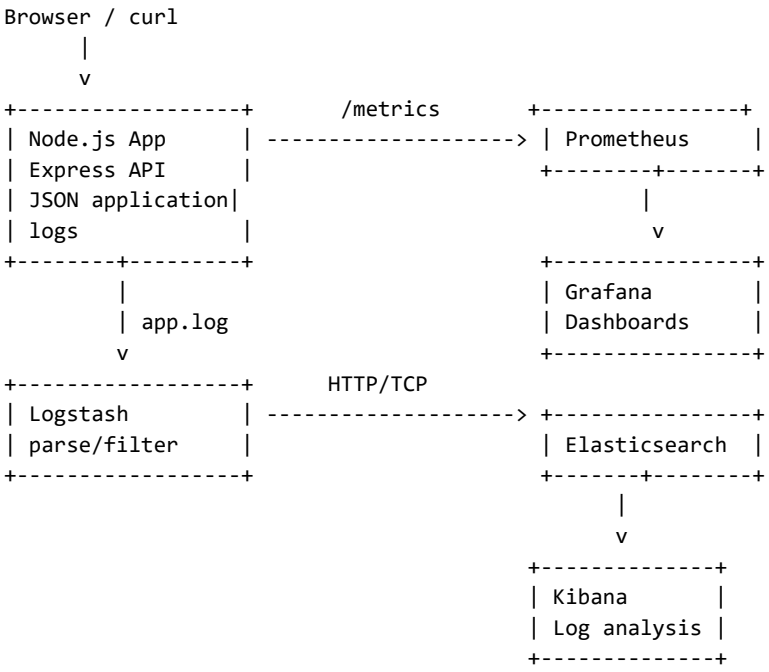
Experiment No.	_____
Suggested Duration	2 hours
Difficulty	Intermediate
Primary Tools	Docker / Docker Compose, Prometheus, Grafana, Elasticsearch, Logstash, Kibana
Application	Node.js Express demo service with Prometheus metrics and JSON logs

Academic focus: Students learn observability as an engineering practice: metrics answer "what is happening?", dashboards help humans interpret it, and centralized logs answer "what happened and why?".

Safety: Run the stack locally. Do not expose Elasticsearch, Grafana, Kibana, or Prometheus directly to the public Internet for this classroom exercise.

1. Experiment Overview

Set up a local observability stack for a deployed web application. The application exposes Prometheus metrics through /metrics and writes structured JSON logs to a host-mounted file. Prometheus scrapes metrics, Grafana visualizes them, Logstash ingests the logs, Elasticsearch indexes them, and Kibana provides log search and analysis.



2. Learning Objectives

- Explain observability and distinguish metrics, logs, and dashboards.
- Deploy a multi-container monitoring/logging stack using Docker Compose.
- Instrument a Node.js application with Prometheus-compatible metrics.
- Configure Prometheus scrape targets and validate target health.
- Create Grafana panels for request rate, error count, latency, and memory usage.
- Ship structured JSON logs through Logstash into Elasticsearch.
- Search and filter logs in Kibana using fields and time ranges.
- Inject controlled failures and correlate metric anomalies with log events.

3. Prerequisites

- Basic Linux/Windows terminal usage and TCP/IP concepts.
- Basic JavaScript/Node.js or willingness to copy and run the provided application.
- Docker Desktop with Docker Compose support, or Docker Engine + Compose plugin.
- VS Code or another text editor.
- At least 4 CPU cores and 6–8 GB RAM available for the ELK stack plus the application.
- Ports available: 3000, 3001, 4000, 5044, 5601, 9200, 9090.

Versioning note: For a teaching lab, pin compatible image versions in your repository rather than relying on “latest” for reproducibility. The examples below intentionally use version placeholders that can be replaced by the lab instructor.

4. Conceptual Background

Metrics: Numeric time-series measurements such as request count, request duration, active connections, memory usage, and error totals.

Logs: Event records describing what an application or component did. Structured JSON makes fields such as level, route, status, and requestId searchable.

Prometheus: A time-series monitoring system that scrapes instrumented targets. Its configuration defines scrape jobs and targets.

Grafana: A visualization and observability UI that connects to data sources such as Prometheus and builds dashboards and alerts.

ELK: Elasticsearch indexes and searches documents; Logstash collects/transforms events; Kibana provides a browser UI for investigation.

5. Files to Create

```
observability-lab/
├── app/
│   ├── package.json
│   └── server.js
├── logs/
│   └── app.log
├── logstash/
│   └── pipeline/
│       └── logstash.conf
├── prometheus/
│   └── prometheus.yml
└── docker-compose.yml
```

6. Step 1 — Build the Demo Application

Create the following files. The service exposes /, /health, /work, /error and /metrics. The /work endpoint adds a small randomized response delay to make latency visible in Grafana.

app/package.json

```
{
  "name": "observability-demo",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "express": "^5.0.0",
    "prom-client": "^15.0.0"
  }
}
```

app/server.js

```
const express = require('express');
const fs = require('fs');
const client = require('prom-client');
```

```
const app = express();
const port = 4000;
const logFile = '/logs/app.log';
fs.mkdirSync('/logs', { recursive: true });

client.collectDefaultMetrics();
const httpRequests = new client.Counter({
  name: 'app_http_requests_total',
  help: 'Total HTTP requests',
  labelNames: ['method', 'route', 'status']
});
const httpDuration = new client.Histogram({
  name: 'app_http_request_duration_seconds',
  help: 'HTTP request duration in seconds',
  labelNames: ['method', 'route', 'status'],
  buckets: [0.05, 0.1, 0.25, 0.5, 1, 2, 5]
});

function writeLog(level, message, extra = {}) {
  const entry = {
    timestamp: new Date().toISOString(),
    level,
    service: 'observability-demo',
    message,
    ...extra
  };
  fs.appendFileSync(logFile, JSON.stringify(entry) + '\n');
  console.log(JSON.stringify(entry));
}

app.use((req, res, next) => {
  const start = process.hrtime.bigint();
  res.on('finish', () => {
    const seconds = Number(process.hrtime.bigint() - start) / 1e9;
    const route = req.route?.path || req.path;
    httpRequests.inc({ method: req.method, route, status: String(res.statusCode) });
    httpDuration.observe({ method: req.method, route, status: String(res.statusCode) }, seconds);
    writeLog(res.statusCode >= 500 ? 'ERROR' : 'INFO', 'request_completed', {
      method: req.method, route, status: res.statusCode, duration_ms: Math.round(seconds * 1000)
    });
  });
  next();
});

app.get('/', (req, res) => res.json({ message: 'Observability lab application is running' }));
app.get('/health', (req, res) => res.json({ status: 'UP' }));
app.get('/work', async (req, res) => {
  const delay = Math.floor(Math.random() * 900) + 100;
  await new Promise(resolve => setTimeout(resolve, delay));
  res.json({ status: 'completed', delay_ms: delay });
});
app.get('/error', (req, res) => {
  writeLog('ERROR', 'controlled_test_error', { errorCode: 'LAB-500' });
  res.status(500).json({ error: 'Controlled lab error' });
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});
```

```
app.listen(port, () => writeLog('INFO', 'server_started', { port }));
```

7. Step 2 — Configure Prometheus

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s

scrape_configs:
  - job_name: 'observability-app'
    static_configs:
      - targets: ['app:4000']
```

Prometheus configuration is YAML. The scrape_configs section defines jobs and targets, while scrape_interval controls how frequently Prometheus collects samples. In this lab, the application container is reachable by the Compose service name app. Prometheus target configuration follows the official Prometheus model.

8. Step 3 — Configure Logstash

```
input {
  file {
    path => "/logs/app.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
    codec => json
  }
}

filter {
  mutate {
    add_field => { "lab" => "observability" }
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "app-logs-%{+YYYY.MM.dd}"
  }
  stdout { codec => rubydebug }
}
```

Design choice: The application writes one JSON event per line to a mounted directory. Logstash tails that file, parses JSON, adds a lab field, and indexes the event in Elasticsearch. This avoids requiring students to understand Docker daemon logging internals in the first exercise.

9. Step 4 — Create Docker Compose Stack

```
services:
  app:
    image: node:22-alpine
    working_dir: /app
    command: sh -c "npm install && node server.js"
```

```
    ports:
      - "4000:4000"
    volumes:
      - ./app:/app
      - ./logs:/logs

  prometheus:
    image: prom/prometheus:<PINNED_VERSION>
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    depends_on:
      - app

  grafana:
    image: grafana/grafana:<PINNED_VERSION>
    ports:
      - "3001:3000"
    depends_on:
      - prometheus

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:<PINNED_VERSION>
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"

  logstash:
    image: docker.elastic.co/logstash/logstash:<PINNED_VERSION>
    volumes:
      - ./logs:/logs:ro
      - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
    depends_on:
      - elasticsearch

  kibana:
    image: docker.elastic.co/kibana/kibana:<PINNED_VERSION>
    environment:
      ELASTICSEARCH_HOSTS: http://elasticsearch:9200
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
```

Instructor note: Replace <PINNED_VERSION> with one tested, mutually compatible ELK release and a tested Grafana/Prometheus release before distributing the lab. Pinning avoids breaking changes between semesters.

10. Step 5 — Start the Environment

1. Open a terminal in the observability-lab directory.
2. Verify Docker is running: docker version
3. Verify Compose: docker compose version
4. Create logs/app.log as an empty file if your OS requires it.

5. Start all services: docker compose up -d
6. Check status: docker compose ps
7. Inspect startup messages: docker compose logs --tail=50

```
# Useful URLs
Application:  http://localhost:4000
Prometheus:   http://localhost:9090
Grafana:      http://localhost:3001
Elasticsearch:http://localhost:9200
Kibana:       http://localhost:5601
```

11. Step 6 — Validate the Monitoring Path

1. Open http://localhost:4000/metrics and confirm Prometheus-format metrics are visible.
2. Open Prometheus → Status → Targets and verify observability-app is UP.
3. In Prometheus Graph, query app_http_requests_total and execute the query.
4. Generate traffic with repeated requests to /, /health and /work.

```
# Linux/macOS/Git Bash example
for i in {1..20}; do curl -s http://localhost:4000/work > /dev/null; done

# PowerShell alternative
1..20 | ForEach-Object { Invoke-WebRequest http://localhost:4000/work | Out-Null }
```

Expected result: The request counter increases, the latency histogram receives observations, and default Node.js runtime metrics are visible because collectDefaultMetrics() was enabled.

12. Step 7 — Configure Grafana Dashboard

1. Open Grafana at http://localhost:3001.
2. Sign in with the credentials shown on the lab image/configuration used by your instructor.
3. Add a Prometheus data source using URL http://prometheus:9090. Use the container DNS name, not localhost, because Grafana runs inside the Compose network.
4. Create a dashboard with the following panels.

Panel	PromQL example	Interpretation
Requests / sec	rate(app_http_requests_total[1m])	Traffic rate
5xx rate	sum(rate(app_http_requests_total{status=~"5.."}[1m]))	Server error rate
P95 latency	histogram_quantile(0.95, sum(rate(app_http_request_duration_seconds_bucket[5m])) by (le))	Tail latency
Memory	process_resident_memory_bytes	Application memory footprint
CPU	rate(process_cpu_seconds_total[1m])	CPU consumption

13. Step 8 — Configure Kibana and Analyze Logs

1. Open http://localhost:5601.
2. Create a data view matching app-logs-*.
3. Select @timestamp as the time field if your Elasticsearch mapping identifies it; otherwise use the event timestamp field available in your environment.
4. Open Discover and inspect log events.

5. Filter by level: ERROR.

6. Filter by route: /work or /error.

7. Search for errorCode: LAB-500.

```
Example investigation questions:
level: ERROR
status: 500
route: "/error"
errorCode: "LAB-500"
```

14. Step 9 — Controlled Failure and Correlation Exercise

Generate normal traffic, then intentionally produce errors and latency. Students must correlate dashboard changes in Grafana with log events in Kibana.

1. Generate 50 requests to /health and /work.

2. Generate 10 requests to /error.

3. Observe the 5xx rate panel in Grafana.

4. Open Kibana and search for level: ERROR.

5. Compare the number and timestamps of metric anomalies with the log events.

6. Write a 5–7 line incident summary identifying symptom, likely cause, evidence, and mitigation.

15. Student Tasks / Deliverables

- Submit the docker-compose.yml, prometheus.yml and logstash.conf.
- Submit a screenshot of Prometheus showing the application target UP.
- Submit a Grafana dashboard with at least four panels.
- Submit a Kibana Discover screenshot showing ERROR events.
- Record the results of the controlled failure exercise.
- Answer the viva questions in the next section.

16. Troubleshooting Guide

Symptom	Likely cause	Action
Prometheus target DOWN	App not listening / wrong service name	docker compose logs app; confirm app:4000 and /metrics
Grafana cannot reach Prometheus	Wrong URL	Use http://prometheus:9090 from Grafana container
Kibana shows no documents	Logstash input or Elasticsearch issue	Check docker compose logs logstash and index existence
ELK containers restart	Insufficient RAM / Elasticsearch bootstrap issue	Increase Docker memory; inspect Elasticsearch logs
No new logs	File not mounted or Logstash cannot read it	Check ./logs mount and permissions; verify app.log changes
Dashboard is empty	Wrong data source/query/time range	Test PromQL in Prometheus first, then use the same query in Grafana

19. Advanced Extensions

- Add Prometheus alert rules for 5xx rate > 5% for 2 minutes.
- Add a /slow endpoint and measure p50, p95 and p99 latency.
- Add requestId propagation and correlate the same request across metrics and logs.
- Replace file-based Logstash input with a container logging pipeline.
- Add Alertmanager and send a notification for high error rate.
- Deploy the application on Kubernetes and monitor pods using Prometheus/Grafana.
- Compare pull-based metrics with OpenTelemetry-based telemetry collection.

20. References / Instructor Reading

- **Prometheus documentation — Configuration and scrape jobs:** https://prometheus.io/docs/prometheus/latest/configuration/configuration/
- **Prometheus documentation — First steps:** https://prometheus.io/docs/introduction/first_steps/
- **prom-client — Prometheus client for Node.js:** https://github.com/siimon/prom-client
- **Grafana documentation — Docker installation and configuration:** https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- **Elastic documentation — Docker / Elasticsearch / Kibana / Logstash:** https://www.elastic.co/guide/en/elasticsearch/reference/current/docker.html

End of Laboratory Manual